Deep Learning review

Manuel

March 10, 2026

1 MLP 2

2 Optimization: Gradient Descent 3

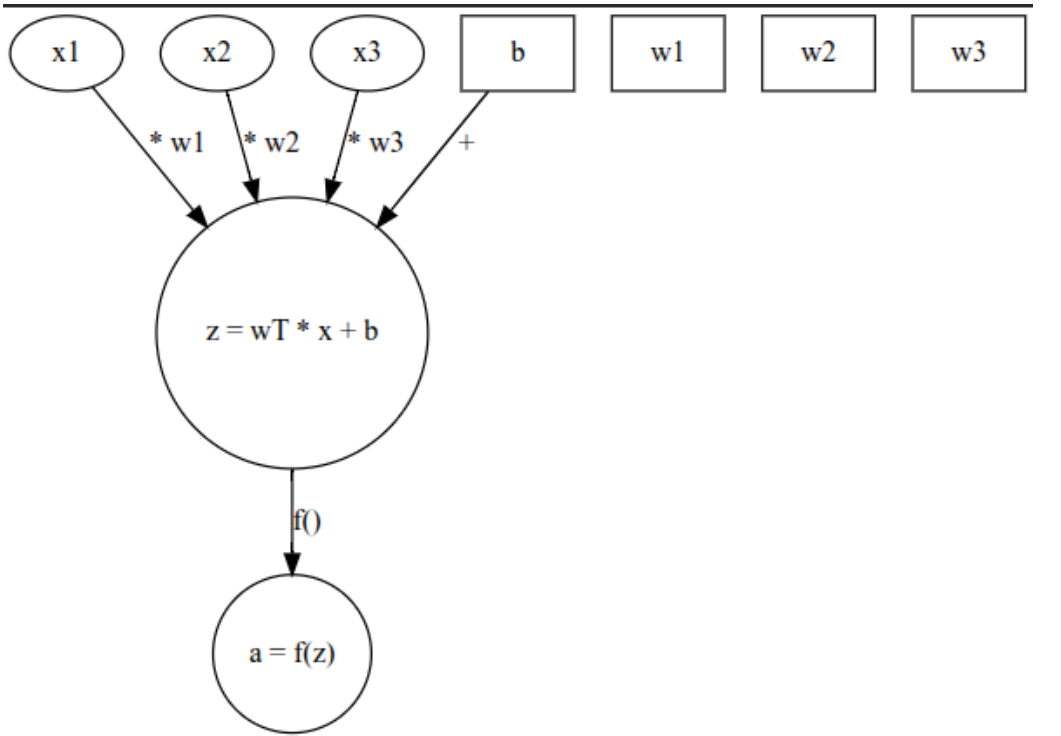
3 Backpropagation 5

4 Recurrent Neural Networks (RNN) 8

5 Long Short-Term Memory (LSTM) 13

6 Transformers 16

MLP



Gradient Descent: The Optimization Engine

The Update Rule

For a set of parameters θ, the update at step t is:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta J(\theta_t)$$

Where:

- η: **Learning rate** (step size).
- $\nabla_\theta J(\theta_t)$: **Gradient vector** (direction of steepest ascent).

Gradient Descent: Pros and Cons

Pros	Cons
Conceptually simple and theoretically grounded in calculus.	Risk of getting stuck in **local minima** or **saddle points**.
Scalable to millions of parameters (via SGD/Adam).	Highly sensitive to the choice of the learning rate (η).
Guaranteed convergence for convex loss functions.	Computationally expensive if the full dataset is used (Batch GD).

Transition to Backpropagation: In deep networks, calculating $\nabla_\theta J$ directly is intractable. We need an efficient way to apply the **Chain Rule** from the output back to the input.

BACKPROPAGATION

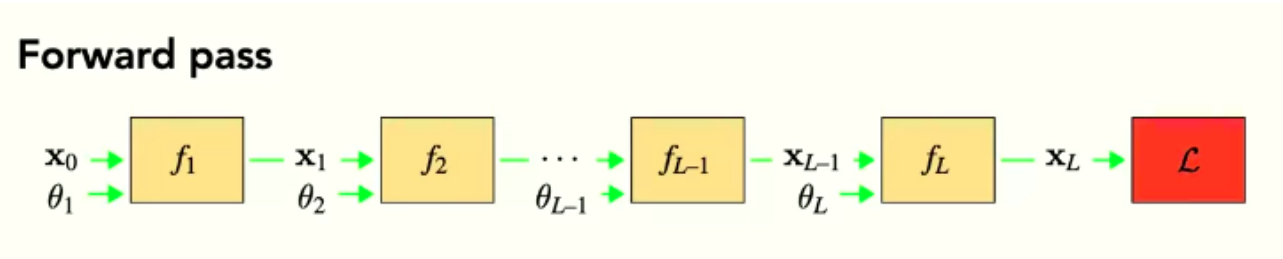


Figure. Caption

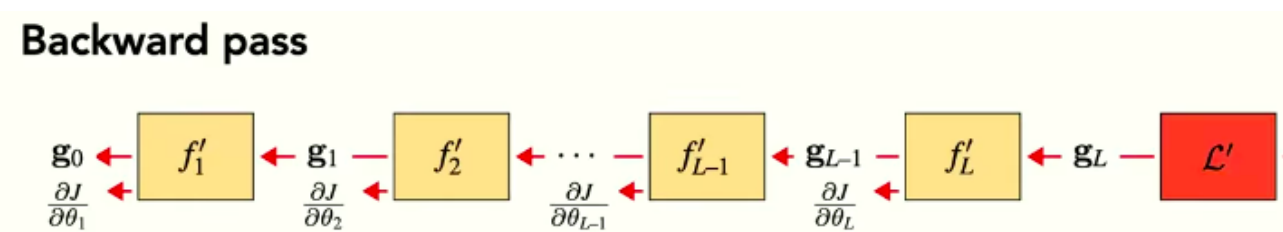


Figure. Caption

Backpropagation

$$
\begin{aligned}
\partial J/\partial\theta_1 &= \partial J/\partial x_L \partial x_L/\partial x_{L-1} \cdots \partial x_2/\partial x_1 \partial x_1/\partial\theta_1 \\
\partial J/\partial\theta_2 &= \partial J/\partial x_L \partial x_L/\partial x_{L-1} \cdots \partial x_3/\partial x_2 \partial x_2/\partial\theta_2 \\
&\vdots
\end{aligned}
$$

Updating the weights to minimize the cost:

$$
\theta^{i+1} = \theta^i - \eta \frac{\partial \boldsymbol{J}^T}{\partial\theta} \tag{1}
$$

BACKPROPAGATION ALGORITHM

Input: parameter vector $\theta = \{\theta_l\}_{l=1}^{L}, f_1, \ldots, f_L, f_1', \ldots, f_L'$, training datapoint $\{\mathbf{x}_0, \mathbf{y}\}$, Loss function $\mathcal{L} : \mathbb{R}^N \to \mathbb{R}$

Output: gradient direction $\frac{\partial J}{\partial \theta} = \left\{ \frac{\partial J}{\partial \theta_l} \right\}_{l=1}^{L}$

Forward pass:

for $l = 1, \ldots, L$ **do**

  $\mathbf{x}_l = f_l(\mathbf{x}_{l-1}, \theta_l)$

Backward pass:

$\mathbf{g}_L = \mathcal{L}'(\mathbf{x}_L, y)$

for $l = L, \ldots, 1$ **do**

  $\mathbf{L}_l = f_l'(\mathbf{x}_{l-1}, \theta_l)$

  $\mathbf{g}_{l-1} = \mathbf{g}_l \mathbf{L}_l^{\mathbf{x}}$

  $\frac{\partial J}{\partial \theta_l} = \mathbf{g}_l \mathbf{L}_l^{\theta}$

Figure. Caption

Recurrent Neural Networks (RNN)

- RNNs are designed to handle sequential data (text, audio, time series).
- Hidden State (h_t): The network maintains an internal "memory" that is updated at each time step.

Mathematical Model

At each time step t, the hidden state h_t and output y_t are calculated as:

$$h_t = \phi(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = \mathrm{f}(W_{hy}h_t + b_y)$$

RNN

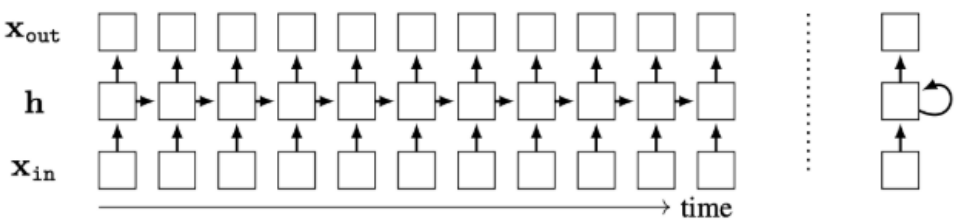


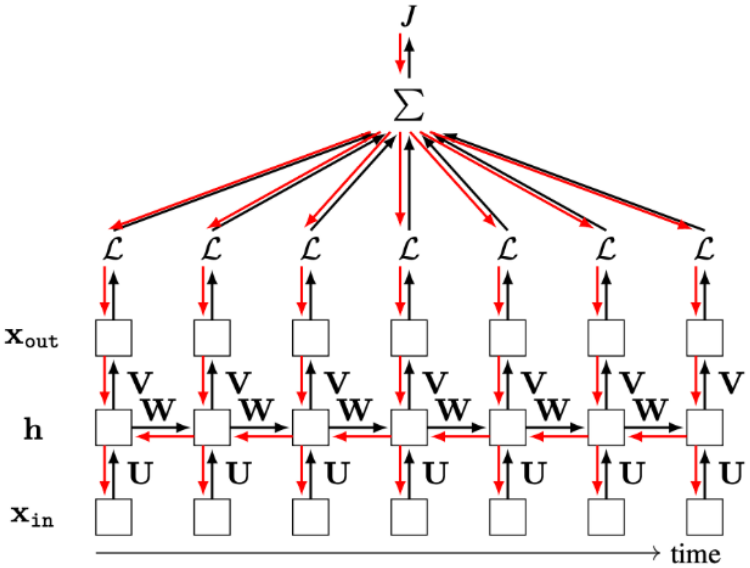
Figure. Caption

RNN

Pros	Cons
Variable Length: Can process sequences of any size.	**Sequential Bottleneck:** Cannot process words in parallel (slow training).
Historical Context: Uses previous information to influence current prediction.	**Short-term Memory:** In practice, they struggle to link information that is far apart.
Parameter Efficiency: Uses the same weights at every step.	**Numerical Instability:** Prone to Vanishing and Exploding gradients.

The Gradient Problem in RNNs

Backpropagation Through Time (BPTT): Gradients are calculated by unfolding the RNN.



Recurrent Neural Networks (RNN)

The Gradient Product

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=t+1}^{T} W_{hh}^T \mathrm{diag}(\sigma'(\ldots))$$

LSTM

- LSTMs introduce the Constant Error Carousel (CEC), implemented via a "Cell State" (c_t).
- Instead of simple multiplication, LSTMs use gates to control information flow

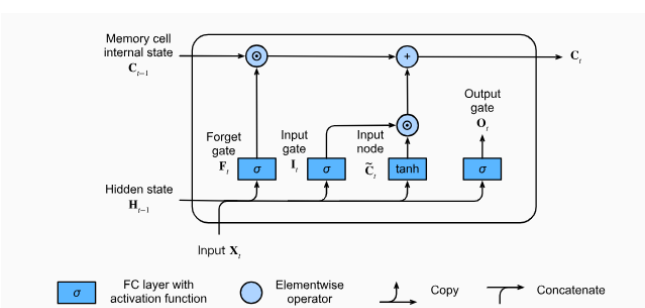


Figure. Caption

Long Short-Term Memory (LSTM)

The Gates

- input node: $\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t\mathbf{W}_{\mathrm{xc}} + \mathbf{H}_{t-1}\mathbf{W}_{\mathrm{hc}} + \mathbf{b}_{\mathrm{c}),}$
- memory cell internal state: $\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t.$
- hidden state: $\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t).$

LSTM: Pros and Cons

Pros

- The additive update of the cell state prevents vanishing gradients.
- Handled complex NLP tasks like translation effectively.

COns

- Like RNNs, they cannot process words in parallel (GPU underutilization)
- 4x more parameters than a vanilla RNN cell
- Still assumes that "closer" words are always more relevant.

Transformers

- The Problem with RNNs/LSTMs: Sequential processing is a bottleneck. We cannot compute word t until $t - 1$ is finished.
- The Solution: Self-Attention. This mechanism allows the model to look at all words in a sequence simultaneously and weigh their relevance to each other.

Self-Attention: Formal Definition

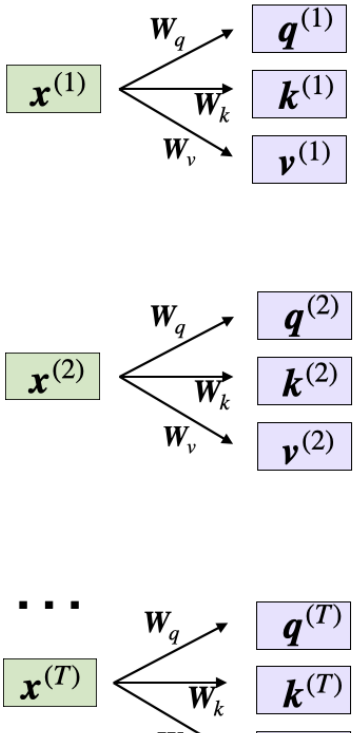
Let $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_T]^\top \in \mathbb{R}^{T \times d}$ be a sequence of n input tokens, where each $\mathbf{x}_i \in \mathbb{R}^d$ is a feature vector.

- **Input:** A set of vectors in a d-dimensional space.
- **Output:** A new set of vectors $\mathbf{z} = [\mathbf{z}_1, \ldots, \mathbf{y}_T]^\top \in \mathbb{R}^{T \times d_v}$.

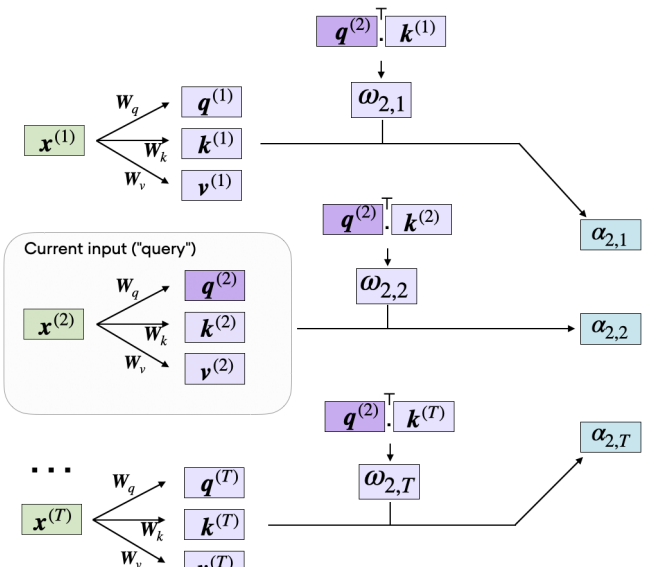
TOKENIZATION

- tokenization: Byte Pair Encoding (BPE) used by GPT (OpenAI), Llama (Meta) and RoBERTa,
- exmaple for ’La inteligencia es asombrosa’: ("GLa", "Ginteligencia", "Ges", "Gasom", "brosa")
- WordPiece : used by BERT,
- example: ("La", "inteligencia", "es", "asom", "##brosa")
- SentencePiece:library that implements BPE or Unigram

Self Attention



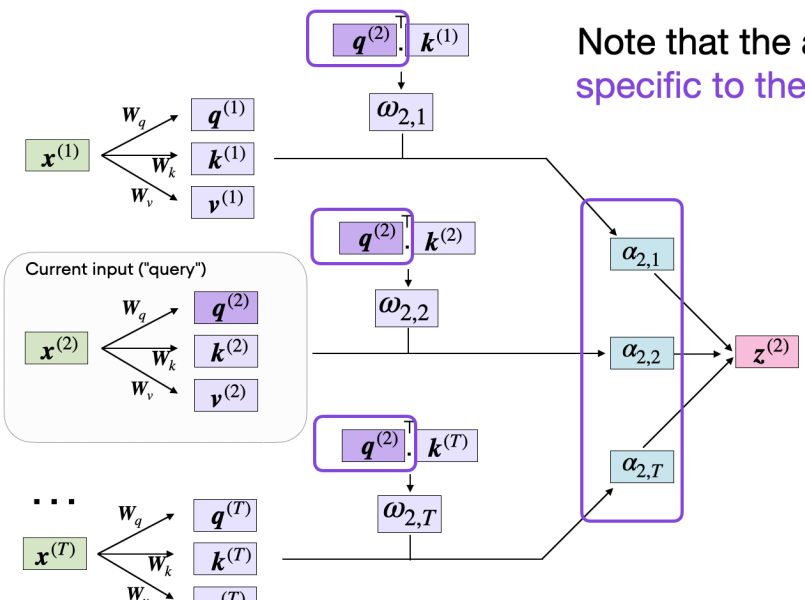
SELF ATTENTION



where $\alpha_{2,i} = \text{softmax}\left(\frac{\omega_{2,i}}{\sqrt{d_k}}\right)$

Figure. Caption

SELF ATTENTION



Note that the attention scores are specific to the current input token

where $z^{(2)} = \sum_{j=1}^{T} \alpha_{2,j} v^{(j)}$

Figure. Caption

Self Atention

The output vector $\mathbf{z}_i$ for the i-th token is a **convex combination** of projected input vectors:

$$\mathbf{z}^{(i)} = \sum_{j=1}^{T} \alpha_{ij} \mathbf{v}^{(j)}$$

where:

- $\alpha_{ij} \in [0, 1]$ are the **Attention Weights** ($\sum_j \alpha_{ij} = 1$).
- $\mathbf{v}^j = \mathbf{W}_V \mathbf{x}^{(j)}$ is a linear transformation of the input.

Positional Encoding

Vector Addition

Let $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n]^\top$ be the input matrix. The encoder receives:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i + \mathbf{p}_i$$

Where:

- $\mathbf{x}_i \in \mathbb{R}^d$: Semantic embedding (the "meaning").
- $\mathbf{p}_i \in \mathbb{R}^d$: Position embedding (the "coordinate").

THE [CLS] TOKEN: SEQUENCE SUMMARY

- After Self-Attention, we have n vectors. How do we get *one* single representation for the whole sentence?
- We add a special dummy token at the very beginning of the sequence, called [CLS].

Sequence Augmentation

If your input is n tokens, you actually feed $n + 1$ to the model:

$$\mathbf{X}_{augmented} = [\mathbf{x}_{CLS}, \mathbf{x}_1, \ldots, \mathbf{x}_n]^\top$$

How [CLS] Works

Final Classification

We take the first output vector and pass it through a simple linear layer:

$$\text{Class} = \text{Softmax}(\mathbf{W}\mathbf{y}_{CLS} + \mathbf{b})$$

Multi-Head Attention: Motivation

- A single attention matrix $A = \text{Softmax}(\frac{QK^T}{\sqrt{d_k}})$ can only focus on one aspect of the relationship between tokens at a time.
- more “head-attention” blocks Allow the model to jointly attend to information from different representation subspaces.

Analogy

In the sentence "The animal didn’t cross the street because it was too tired":

- Head 1: Might associate "it" with "animal" (semantic).
- Head 2: Might associate "it" with "tired" (adjective relationship).

Multi-Head Attention

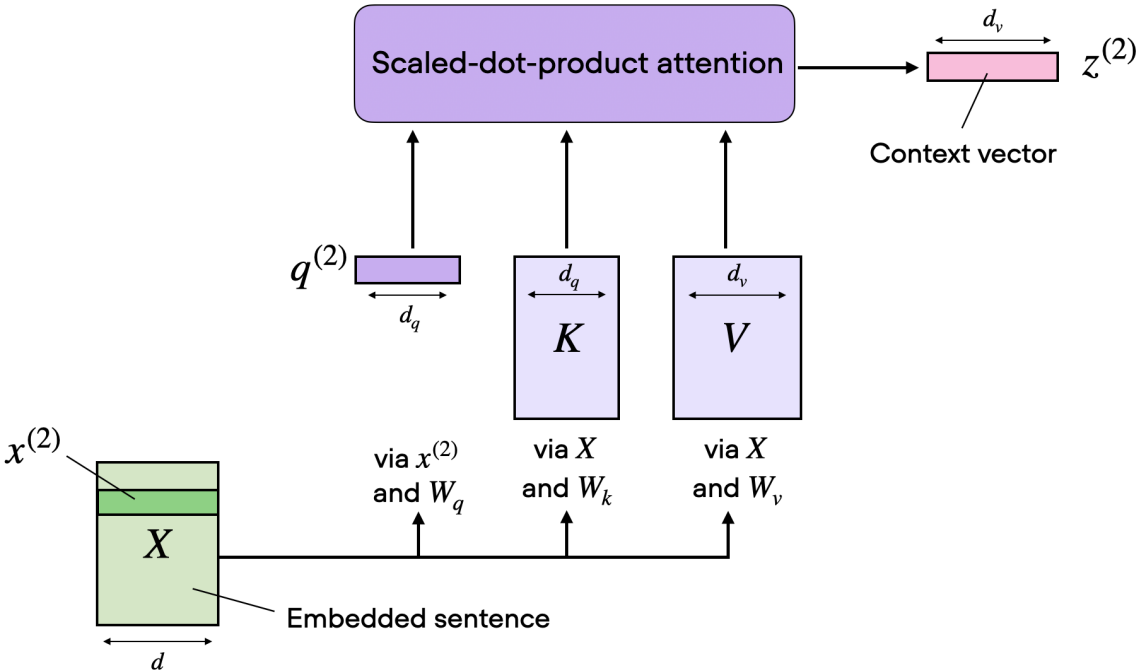
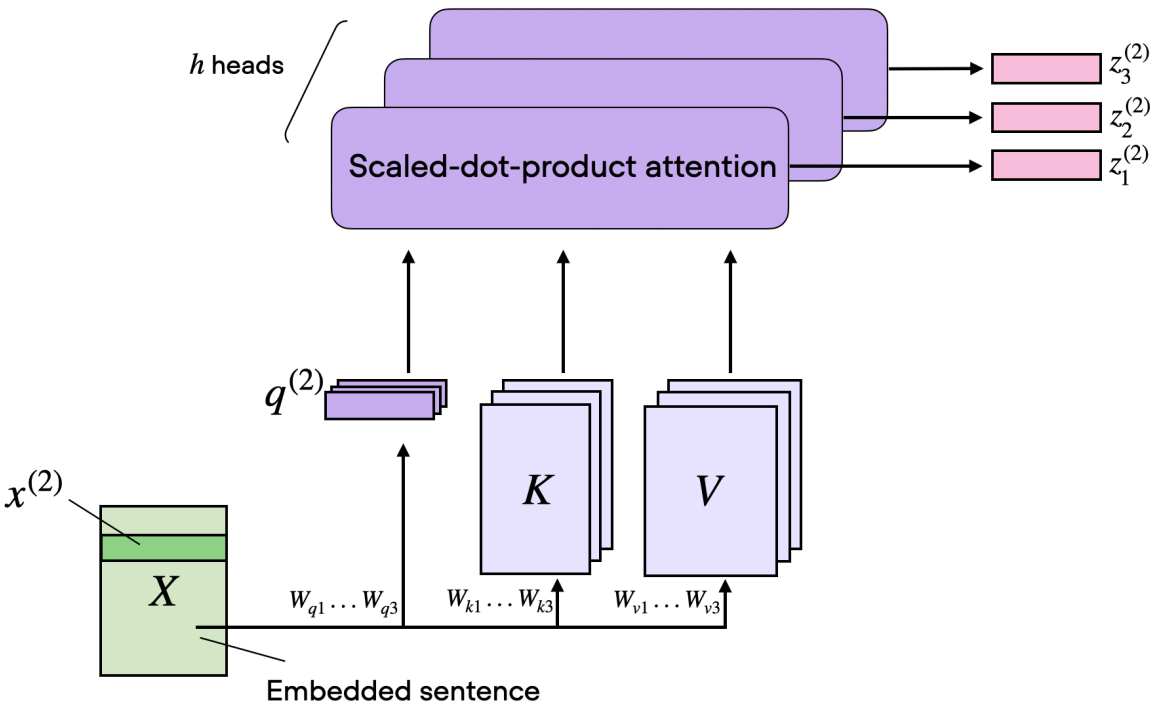


Figure. Caption

Multi-Head Attention



Multi-head Attention

- for example, for BERT embedding dimension: 768
- heads number: 12
- diension for each head: 768/12= 64

Multi-Head Attention: Processing

- Each head h_i computes attention independently:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- **Aggregation:** The outputs are **concatenated**, not summed.
- **Final Projection:** A weight matrix W^O merges the heads back to the original dimension d_{model}.

The Formula

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

Cross-Attention: The Asymmetric Mechanism

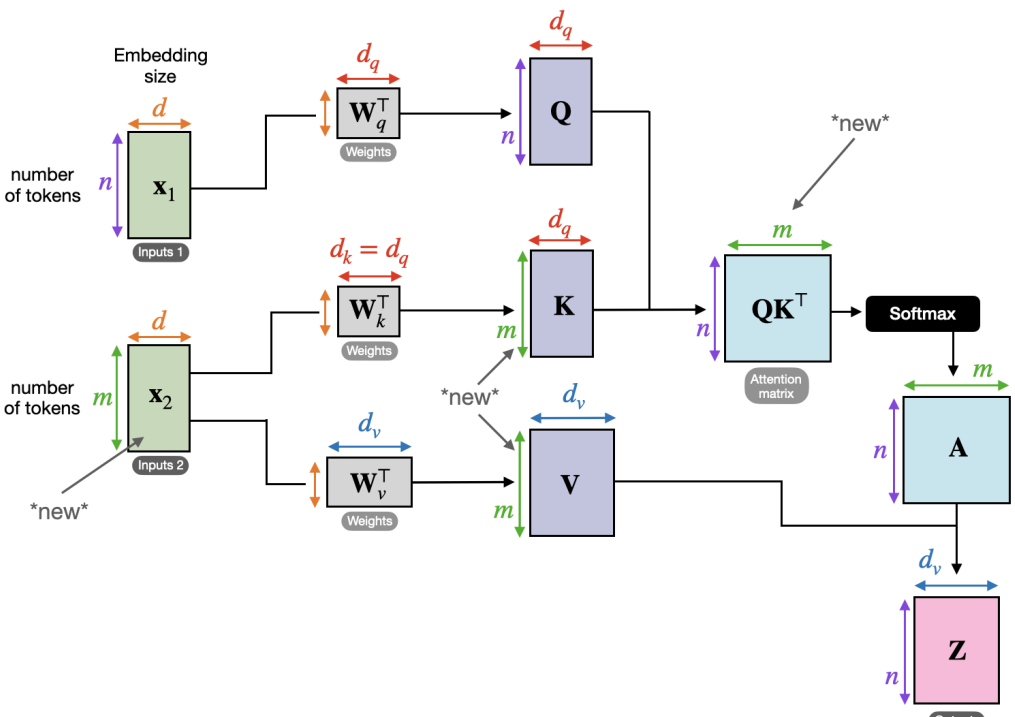
- Self-Attention: Q, K, V are derived from the *same* input sequence $\mathbf{X}$.
- Cross-Attention: Queries come from sequence A, while Keys and Values come from sequence B.

The Objective

It allows one sequence to interrogate another.

- Translation: A Decoder token (Target language) looks for relevant information in the Encoder's output (Source language).

Cross Attention



Cross-Attention: Matrix Dynamics

Let $\mathbf{X}_1 \in \mathbb{R}^{n \times d}$ (e.g., Decoder) and $\mathbf{X}_2 \in \mathbb{R}^{m \times d}$ (e.g., Encoder).

Linear Projections

The mechanism computes:

- $\mathbf{Q} = \mathbf{X}_1 \mathbf{W}_Q^\top \in \mathbb{R}^{n \times d_q}$
- $\mathbf{K} = \mathbf{X}_2 \mathbf{W}_K^\top \in \mathbb{R}^{m \times d_k}$
- $\mathbf{V} = \mathbf{X}_2 \mathbf{W}_V^\top \in \mathbb{R}^{m \times d_v}$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{2}$$

The resulting attention matrix $\mathbf{A}$ has dimensions $n \times m$.

Encoder-Decoder: Information Flow

- **Encoder Output:** Provides the *Memory* (K, V). These are high-level features of the source text.
- **Decoder Query:** Generates a *Query* (Q) from its current state.
- **The Result:** The Decoder "attends" to the relevant parts of the Encoder's memory.

Encoder-Decoder

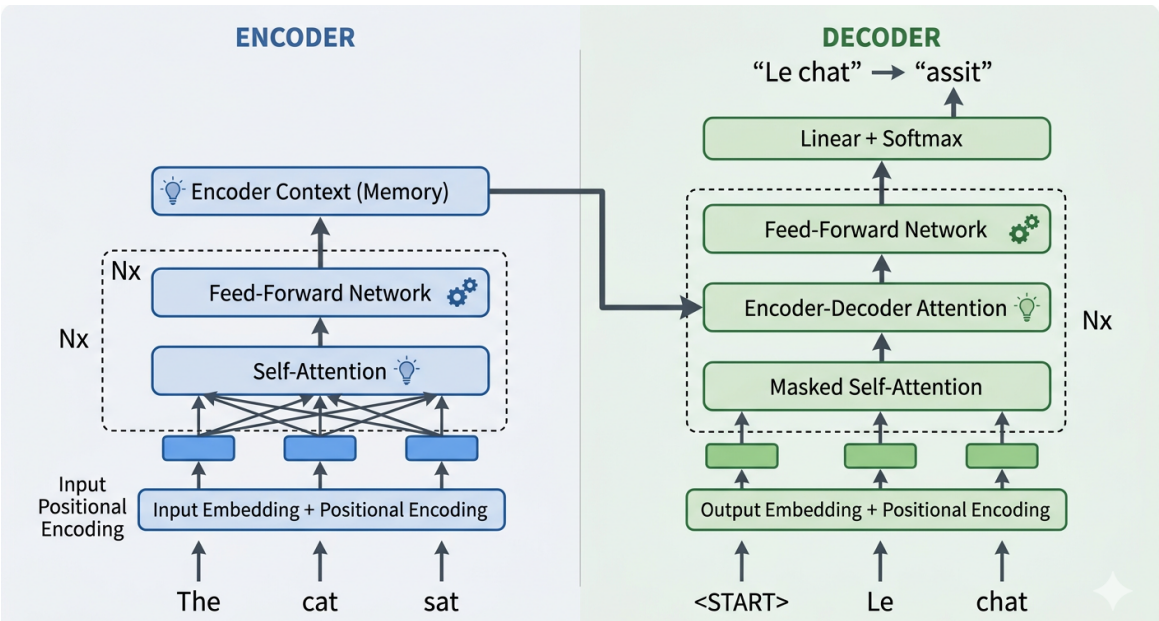


Figure. Caption

Transformers: Pros and Cons

Pros	Cons
Parallelization: Unlike RNNs, the entire sequence is processed at once. No sequential bottleneck.	**Quadratic Complexity:** The self-attention matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ scales $O(n^2)$ with sequence length n.
Global Receptive Field: Any two tokens i, j interact directly in one layer, regardless of distance.	**Lack of Structural Bias:** No inherent order; requires **Positional Encodings** to understand topology.
Multi-Head Flexibility: Can learn different projections (semantic, syntactic) in parallel subspaces.	**Data Hungry:** Needs massive datasets to learn the relationships that RNNs/CNNs assume by design.

References I